AI-100 Midterm Project

(Using Deep Learning to Predict Success Rates for Leetcode Problems)

Professor Wenpeng Yin

AI-100

Student Name: Ojas Hatolkar

01 March 2026

Introduction:

The goal of this project is to use deep learning, specifically a Multi-Layer Perceptron (MLP) , in order to predict whether specific Leetcode problems would have high success rates or low success rates (classification). This goal is the same as that one for Homework 1, where I used the same dataset and applied logistic regression instead of deep learning.

Process (briefly):

I first downloaded the data as a csv file. Then (with the assistance of ChatGPT), I used a Jupyter notebook to write Python code that imported pandas and numpy and preprocessed the data from the initial csv file before splitting the data into training, testing, and validating sets.

Then, I imported tensorflow and keras (from tensorflow). I subsequently built the MLP model by creating two hidden layers and one output layer. Then I trained the model using the training data set from earlier and evaluated the loss and accuracy of this model. Then, using matplotlib.pyplot, I created graphs showing the training curves for this evaluation.

Problem Definition and Dataset Curation:

For this assignment, the selected problem was to apply classification (specifically binary classification) Leetcode problems based on success rate (High and Low) using difficulty level, total number of submissions, and likes as the parameters. A value of 0 would be associated with a low success rate, and a value of 1 with a high success rate. I used the median success rate as the threshold to divide high and low success rates.

The dataset is the same as the one that I used in Homework 1 (link to dataset: https://www.kaggle.com/datasets/manthansolanki/leetcode-questions). It contains various parameters for each Leetcode question, so some preprocessing was required before continuing.

Preprocessing required removing unneeded parameters and duplicates so that the only parameters remaining would be difficulty level, likes, total submissions, and success rate. The data also had to be converted to numerical data, normalized, and scaled from 0 to 1. More specifically, difficulty level (which was categorical data) had to be converted from low, medium, and high to values of 0, 0.5, and 1. The success rate, likes, and number of total submissions (which were numerical data) also had to be converted to a scale of 0 to 1. Then, the data had to be split into sets for training, validation, and testing. These steps were necessary because the data had to be complete, standardized, and in a form that the deep learning model could understand.

Deep Learning Model:

The deep learning model selected for this specific classification problem was multi-layer perceptron (MLP). It was selected because it is well suited for numeric feature vectors that align with the dataset, which uses tabular numerical data. Generally, CNNs and RNNs are more

suitable for images or sequential text respectively, so an MLP was the best choice for this scenario as this data was numerical as opposed to spatial or sequential.

This MLP uses two hidden layers with 32 and 16 neurons. The ReLU function is used in the hidden layers for efficiency. The sigmoid function is used for the output layer in order to apply binary classification and select high or low success rate by mapping output to probabilities between 0 and 1. The loss function for this MLP was the binary cross entropy function, and the optimizer was the Adam Optimizer.

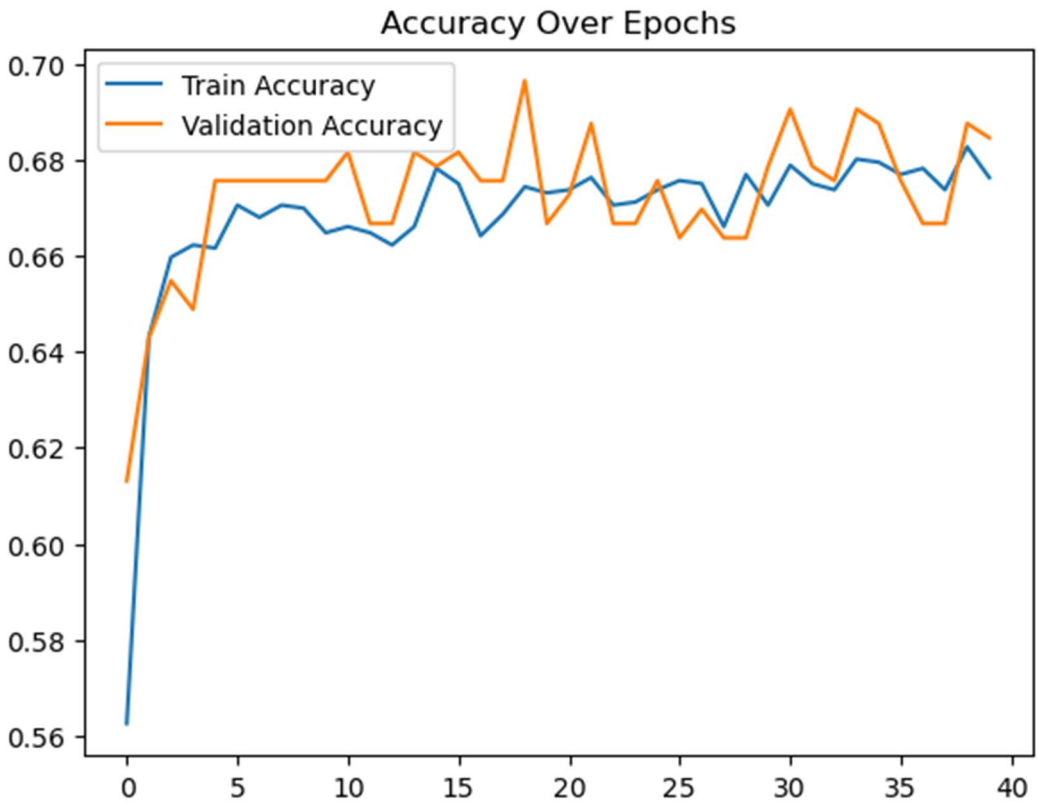
This MLP used 40 epochs, meaning that the deep learning model went through the entire dataset 40 times, processing each data point and applying the loss function every time. The selected batch size was 32, and validation accuracy and loss were monitored.

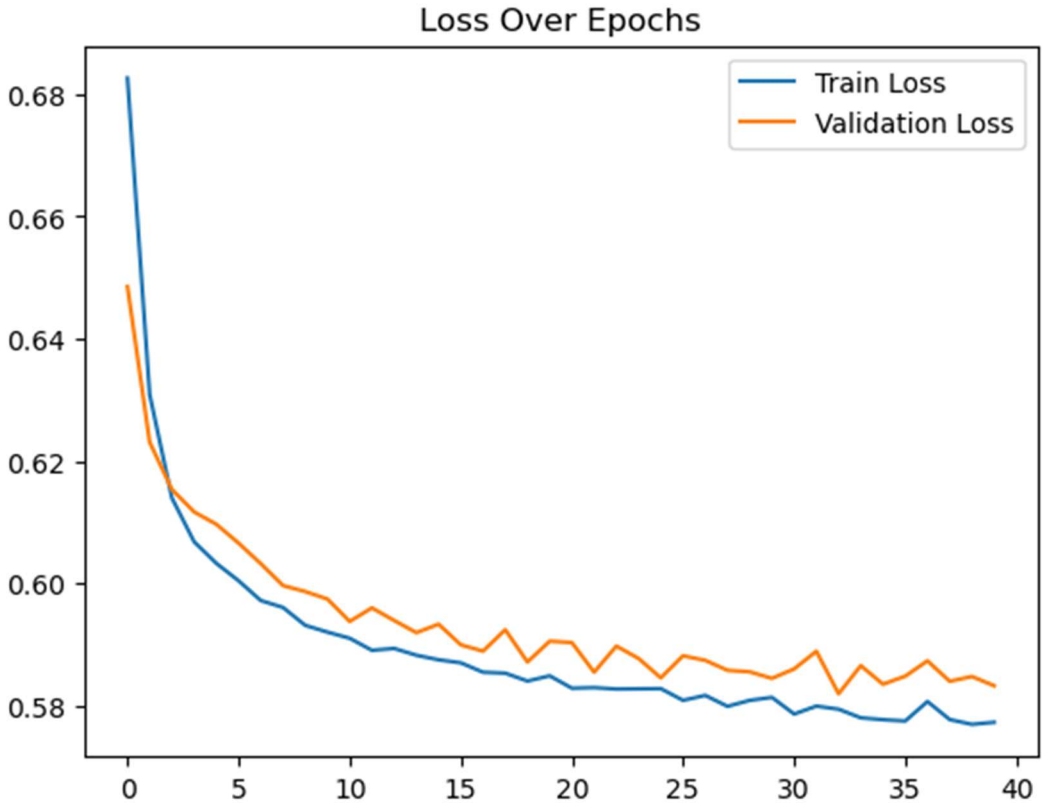
Results:

The MLP had a test accuracy of roughly 66.071%, which was slightly higher than the accuracy of 63% that the logistic regression model from Homework 1 had. This means that the MLP was somewhat accurate in predicting whether a problem would have a high or low success rate, but the results did not have a high level of accuracy. The MLP was slightly more accurate than the logistic regression model. However, this was not a sizeable difference, suggesting that an MLP might not be the ideal approach to this type of classification problem.

The Training and Validation curves generally aligned for both accuracy and loss as shown in the charts below:

**Training curve graphs are on the next page.*


Accuracy Over Epochs
Train Accuracy
Validation Accuracy
0.70
0.68
0.66
0.64
0.62
0.60
0.58
0.56
0
5
10
15
20
25
30
35
40


Loss Over Epochs
Train Loss
Validation Loss
0.68
0.66
0.64
0.62
0.60
0.58
0
5
10
15
20
25
30
35
40

These curves aligned closely, suggesting little to no overfitting, meaning that the MLP did not significantly capture noise and outliers from the training data and can therefore be applied more generally.

Lessons Learned & Experience Gained

Through this project, I became more familiar with using pandas, numpy, and matplotlib (all of which were used in Homework 1) and was introduced to tensorflow.

I also learnt more about preprocessing and how to use built-in functions and libraries (such as StandardScaler) in order to make preprocessing easier. Previously, I had written a separate Python script that would go through the csv file that I had downloaded, preprocess it, and then create a new csv file with the preprocessed data. This was less efficient.

Additionally, I better understood the process of implementing deep learning and multi-layer perceptrons through firsthand experience. I understood the importance of analyzing loss and accuracy. I was able to see how the conceptual understanding of layers translates to a real MLP. I also better understood the importance of optimizing the model for loss.

Finally, the relatively small difference between the accuracies for the logistic regression model and the MLP model may indicate that while deep learning has many strengths and often has higher accuracy than traditional machine learning, it may not always present a significant advantage over simpler models. Depending on individual scenarios, simpler models can still remain competitive on structured tabular data, such as the data for this project.